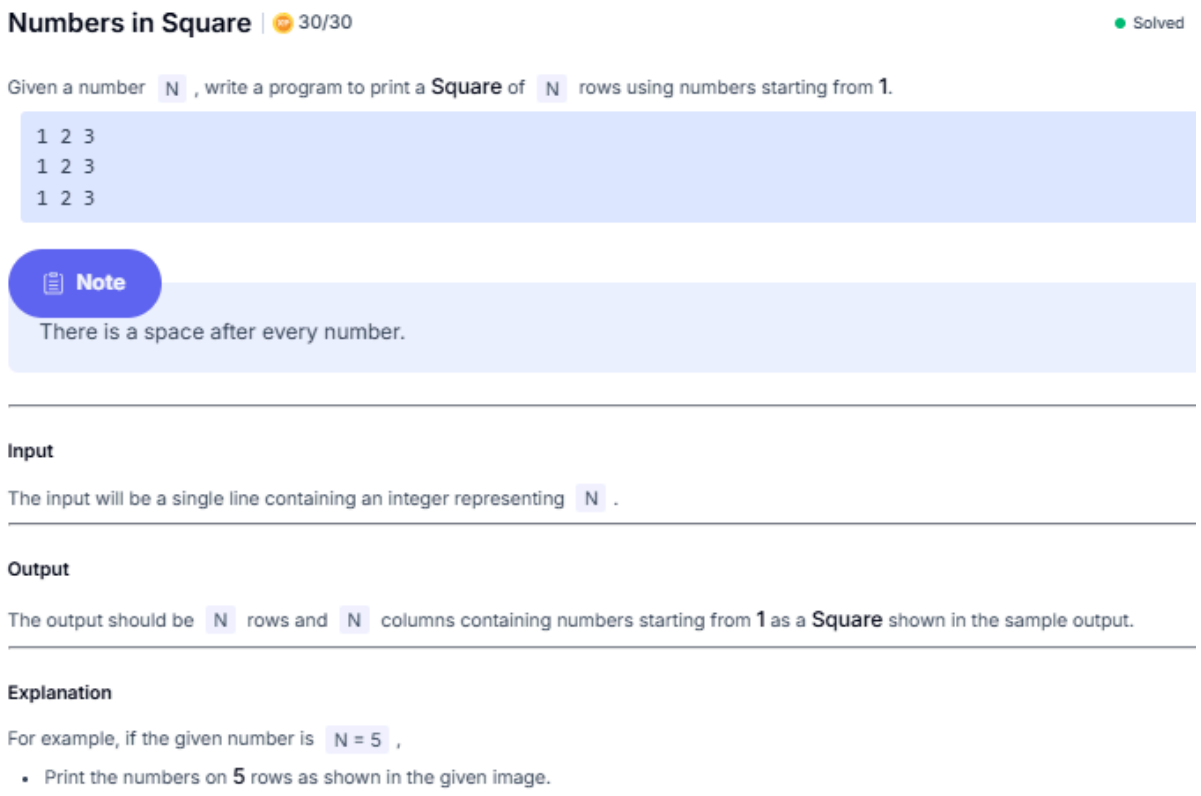
cp_13a

1



Numbers in Square | 30/30

Solved

Given a number `N` , write a program to print a **Square** of `N` rows using numbers starting from **1**.

```
1 2 3
1 2 3
1 2 3
```

Note

There is a space after every number.

Input

The input will be a single line containing an integer representing `N` .

Output

The output should be `N` rows and `N` columns containing numbers starting from **1** as a **Square** shown in the sample output.

Explanation

For example, if the given number is `N = 5` ,

- Print the numbers on **5** rows as shown in the given image.

code:

```
n=int(input())
for i in range(n):
    for j in range(1,n+1):
        print(j,end=" ")
    print()
```

2

Numbers in Right Angled Triangle | 30/30

Solved

Given a number `N` , write a program to print a **Right Angled Triangle** of `N` rows using numbers starting from **1**.

```
1
1 2
1 2 3
```

Note

There is a space after every number.

Input

The input will be a single line containing an integer representing `N` .

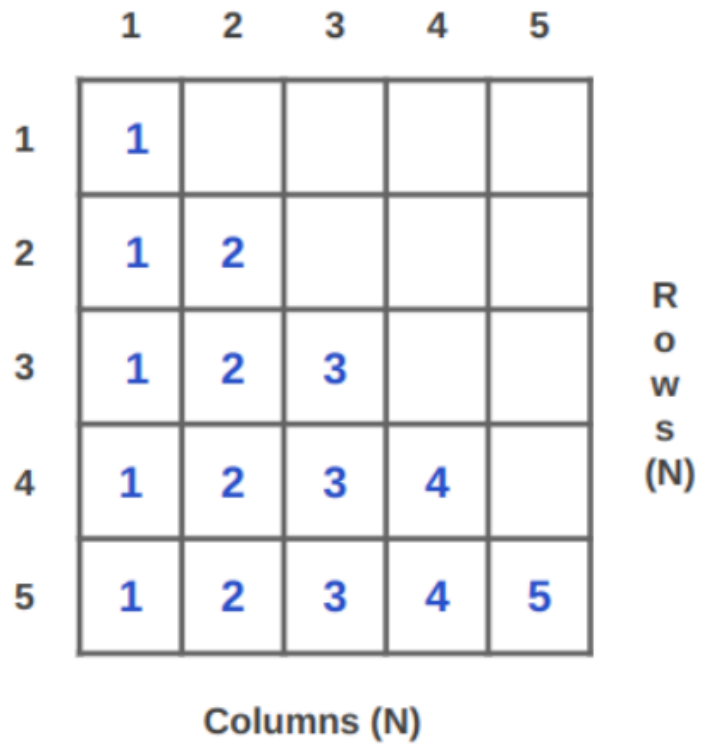
Output

The output should be `N` rows containing numbers from **1** as a **Right Angled Triangle** shown in the sample output.

Explanation

For example, if the given number is `N = 5` ,

- Print the numbers on **5** rows as shown in the given image.



code:

```
num=int(input())
for i in range(1,num+1):
```

```
    for j in range(1,i+1):
        print(j,end=" ")
    print()
```

3

Numbers in Rectangle | 30/30

Solved

Easy

Given two numbers `M` and `N`, write a program to print a **Rectangle** of `M` rows and `N` columns using numbers starting from **7**.

```
7 8 9 10
7 8 9 10
7 8 9 10
```

Note

There is a space after every number.

Input

The first line of input contains an integer representing `M`.
The second line of input contains an integer representing `N`.

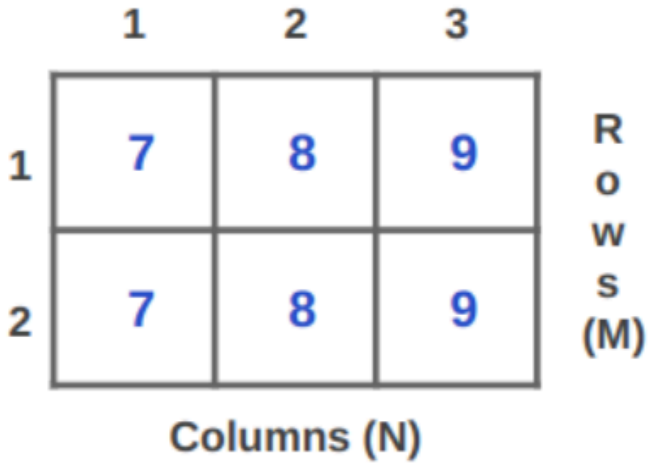
Output

The output should be `M` rows and `N` columns containing numbers as a **Rectangle** starting from **7**, as shown in the sample output.

Explanation

For example, if the given numbers are `M = 2` and `N = 3`,

- Print the numbers on **2** rows and **3** columns starting from **7** as shown in the given image.



code:

```
num=int(input())
n=int(input())
for i in range(num):
    for j in range(7,7+n):
        print(j,end=" ")
    print()
```

4

Numbers in Pyramid | 35/35

Solved

Easy

Given two numbers `S` and `N`, write a program to print a **Pyramid** of `N` rows using numbers starting from `S`.

```
   6
  6 7
 6 7 8
6 7 8 9
```

Input

The first line of input contains an integer representing `S`.
The second line of input contains an integer representing `N`.

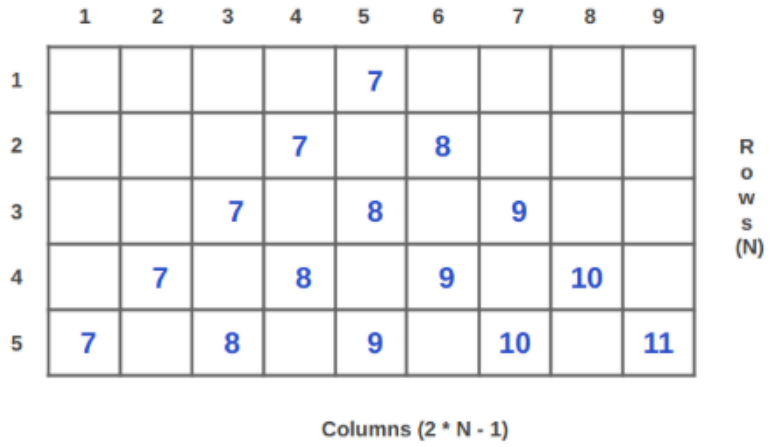
Output

The output should be `N` rows containing numbers starting from `S` as a **Pyramid** shown in the sample output.

Explanation

For example, if the given numbers are `S = 7` and `N = 5`,

- Print the numbers starting from **7** on **5** rows as shown in the given image.



Note

In the given image,

- The Empty box indicates a space (" ").
- The Number in the box indicates a number ("7 ").

code:

```
s=int(input())
n=int(input())
```

```
for i in range(1,n+1):
    print(" "*(n-i),end="")

    for j in range(s,s+i):
        row=str(j)
        print(row,end=" ")

    print()
```

```
5
```

Numbers in Square - 2 | 30/30

Easy

Given a number `N` , write a program to print a **Square** of `N` rows using numbers starting from `N` to `1` .

```
4 3 2 1
4 3 2 1
4 3 2 1
4 3 2 1
```

Note

There is a space after every number.

Input

The input will be a single line containing an integer representing `N` .

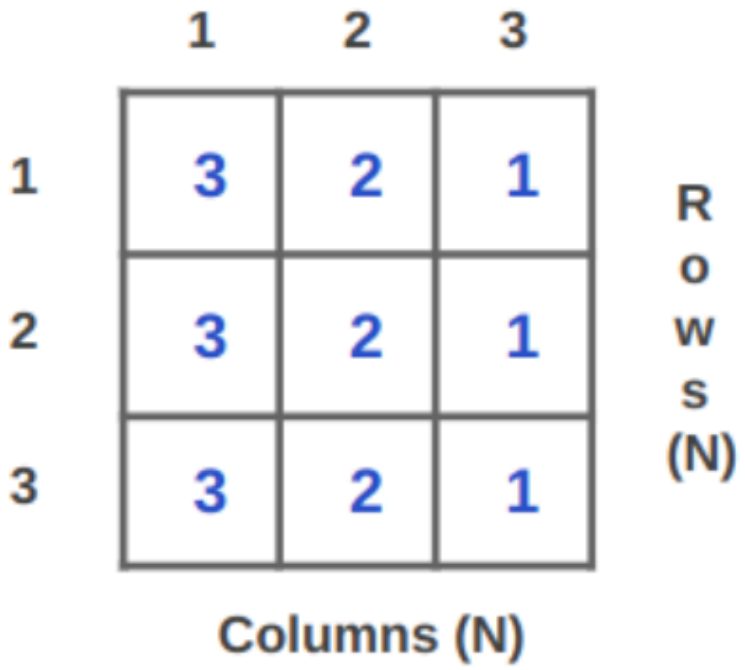
Output

The output should be `N` rows containing numbers from `N` to `1` as a **Square** shown in the sample output.

Explanation

For example, if the given number is `N = 3` ,

- Print the numbers on **3** rows as shown in the given image.



code:

```
n=int(input())
for i in range(1,n+1):
    for j in range(n):
        print((n-j),end=" ")
    print()
```

6

Numbers in Right Angled Triangle - 2 | 35/35

Easy

Given a number `N` , write a program to print a **Right Angled Triangle** of `N` rows using numbers starting from **1**.

```
    1
  2 1
3 2 1
```

Note

There is a space after every number.

Input

The input will be a single line containing an integer representing `N` .

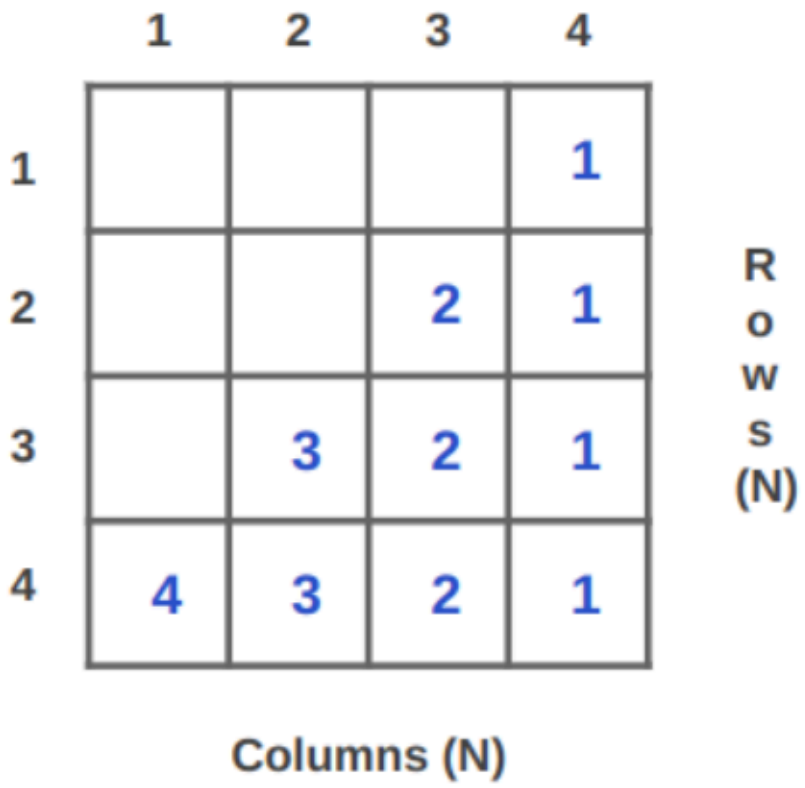
Output

The output should be `N` rows containing numbers as a **Right Angled Triangle** shown in the sample output.

Explanation

For example, if the given number is `N = 4` ,

- Print the numbers on **4** rows as shown in the given image.



code:

```
n=int(input())
N=n
for i in range(1,n+1):

    spaces=" "*((2*N)-2)
    print(spaces,end="")
    for j in range(i):
        print(i-j,end=" ")
    N-=1
    print()
```

7

Armstrong Numbers from 1 to N | 40/40

Solved

Easy

Given a number `N` , write a program to print **Armstrong numbers** from **1** to **N** each on a new line.

Note

A number `N` is an Armstrong Number, if the sum of each digit to the power of `L` is equal to `N` . Where `L` is the count of digits in `N` .

For example if the given number `N = 153` ,

- There are **3** (`L = 3`) digits in **153** are **1**, **2**, **3** .
- The sum of each digit in **153** to the power of **3** is,
- Sum = $(1^3)+(5^3)+(3^3) = 1 + 125 + 27 = 153$

The sum of each digit in **153** to the power of **3** is equal to **153**.

So, **153** is an **Armstrong Number**.

Input

The input will be a single line containing an integer representing `N` .

Output

The output should contain the **Armstrong numbers** from **1** to `N` , each on a new line.

Explanation

For example, if the given number is `N = 200` ,

- The **Armstrong numbers** from **1** to **200** are **1, 2, 3, 4, 5, 6, 7, 8, 9, 153**.
  - There is only **1** digit in **1** is **1**.
  - The sum of each digit in **1** to the power of **1** is **1**. (1^1)
  - The sum of each digit in **1** to the power of **1** is equal to the number **1**.

  So, the number **1** is an **Armstrong number**.
  - There are **3** digits in **153** are **1**, **5** and **3**.
  - The sum of each digit in **153** to the power of **3** is **153**. ($1^3+5^3+3^3$).
  - The sum of each digit in **153** to the power of **3** is equal to the number **153**.

  So, the number **153** is an **Armstrong number**.

code:

```
n=int(input())
for i in range(1,n+1):
    result=0
    str_i=str(i)
```

```
    for j in range(len(str_i)):
        ij=int(str_i[j])
        result+=ij**len(str_i)

    if result==i:
        print(i)
```

8

Prime Numbers from 1 to N | 30/30

Given a number `N` , write a program to print all the **Prime Numbers** from **1** to `N` .

Note

A **Prime Number** is a positive integer greater than **1** that has no factors other than **1** and itself.

Input

The input will be a single line containing an integer representing `N` .

Output

The output should contain the **Prime Numbers** from **1** to `N` , each on a new line.

Explanation

For example, if the given number is `N = 10` ,

- The numbers from **1** to **10** are **1, 2, 3, 4, 5, 6, 7, 8, 9,** and **10**.
- The numbers that has no factors other than **1** and itself from **1** to **10** are **2, 3, 5,** and **7**. The output should be

```
2
3
5
```

code:

```
n=int(input())
for i in range(2,n+1):
    no_of_factors=0
    for j in range(2,i):
        if i%j==0:
            no_of_factors+=1
```

```
    if no_of_factors==0:
        print(i)
```

9

Prime Numbers from M to N | 30/30

Solved

Easy

Given two numbers `M` and `N` . Write a program to print all the **Prime Numbers** in the range from `M` to `N` . If no **Prime Numbers** present in the given range print **No Prime Numbers Found**.

Note

A **Prime Number** is a positive integer greater than **1** that has no factors other than **1** and itself.

Input

The first line of input contains an integer representing `M` .
The second line of input contains an integer representing `N` .

Output

The output should be a single line containing the **Prime Numbers** from `M` to `N` separated by a space. If no prime numbers are present in the given range then **No Prime Numbers Found** should be printed.

Explanation

For example, if the given numbers are `M = 3` and `N = 15` ,

- The numbers from **3** to **15** are **3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,** and **15**.
- The numbers that has no factors other than **1** and itself from **3** to **15** are **3, 5, 7, 11,** and **13**.
- There is a space after each prime number.

The output should be **3 5 7 11 13**.

For example, if the given numbers are `M = 20` and `N = 22` ,

- The numbers from **20** to **22** are **20, 21,** and **22**.
- There is no numbers that has no factors other than **1** and itself from **20** to **22**.

The output should be, **No Prime Numbers Found**.

code:

```
m=int(input())
n=int(input())
prime_count=1
```

```
for i in range(m,n):
    no_of_factors=0
    if i==1:
        continue
        """skip the iteration when i=1"""

    for j in range(2,i):
        if i%j==0:
            no_of_factors+=1

    if no_of_factors<1:
       print(i,end=" ")
       prime_count+=1

if prime_count==1:
    print("No Prime Numbers Found")
```

10

Numbers in Rectangle - 2 | 35/35

Solved

Given two numbers `M` and `N`, write a program to print a **Rectangle** of `M` rows and `N` columns using numbers starting from **1**.

```
1 2 3 4
5 6 7 8
```

Note

There is a space after every number.

Input

The first line of input contains an integer representing M.
The second line of input contains an integer representing N.

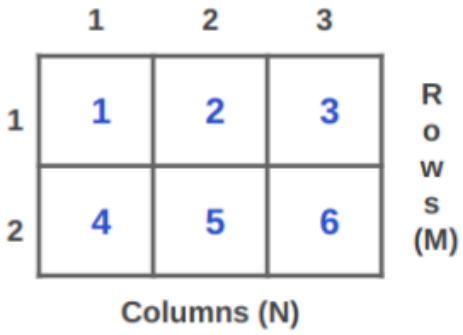
Output

The output should be `M` rows and `N` columns containing numbers as a **Rectangle** shown in the sample output.

Explanation

For example, if the given numbers are `M = 2` and `N = 3`,

- Print the numbers on **2** rows and **3** columns as shown in the given image.



Note

In the given image, the Number in box indicates the number and a space ("1 ")

code:

```
m=int(input())
n=int(input())
k=1
for i in range(m):
    for j in range(n):
        print(k,end=" ")
```

```
            k+=1
        print()
```

11

Numbers in Inverted Right Angled Triangle | 30/30 ● Solved

Given a number `N` , write a program to print an **Inverted Right Angled Triangle** of `N` rows using numbers starting from **1**.

```
1 2 3 4
1 2 3
1 2
1
```

Note

There is a space after every number.

Input

The input will be a single line containing an integer representing `N` .

Output

The output should be `N` rows containing numbers starting from **1** as an **Inverted Right Angled Triangle** shown in the sample output.

Explanation

For example, if the given number is `N = 5` ,

- Print the numbers on **5** rows as shown in the given image.

code:

```
n=int(input())
for i in range(1,n+1):
    for j in range(1,(n+2)-i):
        print(j,end=" ")
    print()
```

12

Half Pyramid - 2 | 35/35

Solved

Write a program to print the numbers in the given N number of rows in the following half pyramid pattern.

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Input

The input will be a single line containing a positive integer (N).

Output

The output should be N rows containing numbers in the half pyramid pattern.
Note: There is a space after each number.

Explanation

For example, if the given number is 5, your code should print the consecutive numbers in the five rows in the half pyramid pattern.

Sample Input 1

```
5
```

Sample Output 1

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

code:

```python
n=int(input())
k=1
for i in range(1,n+1):
    for j in range(1,i+1):
        print(k, end=" ")
        k+=1
    print()
```

13

Prime Numbers from M to N | 30/30

Solved

Given two integers M and N, write a program to print all the prime numbers from M to N.

Input

The first line of input will contain a positive integer (M).
The second line of input will contain a positive integer (N).

Output

The output should be containing all the prime numbers from M to N, each in a new line.

Explanation

For example, if the given numbers are M is 5 and N is 11, as all the prime numbers from 5 to 11 are 5, 7, and 11. So the output should be

```
5
7
11
```

Similarly, if the given numbers are M is 18 and N is 40, as all the prime numbers from 18 to 40 are 19, 23, 29, 31,and 37. So the output should

```
19
23
29
31
37
```

Sample Input 1

```
5
11
```

Sample Output 1

```
5
7
11
```

code:

```
m=int(input())
n=int(input())

for each in range(m,n+1):
    no_of_factors=0

    for j in range(2,each):
        if each%j==0:
            no_of_factors+=1
```

```
    if no_of_factors==0:
        print(each)
```